

PocketPaw - Project Overview & Issue #494 Implementation Report

Project Overview

PocketPaw is an advanced AI agent framework built to provide intelligent tool orchestration and secure, autonomous execution. It uses a `ToolRegistry` to manage the lifecycle and execution of various utilities extending the agent's capabilities. With comprehensive audit logging, multi-layered security (e.g., prompt injection scanning), and modular tool definitions, PocketPaw aims for production-grade reliability.

The Problem: Issue #494

In the original implementation, the `ToolRegistry.execute()` method awaited `tool.execute(**params)` **without any timeout boundary**. This introduced a critical reliability flaw: if a tool (like a web scraper, a long-running subprocess, or a network request) hung indefinitely, the entire agent session would be blocked forever. There was no error thrown, no log emitted, and no way to recover without restarting the agent process.

Changes Made From the Beginning

To fix this, a comprehensive, per-tool timeout mechanism was engineered:

1. Configuration & Settings (`src/pocketpaw/config.py`)

- Added a `tool_timeout` setting to the global `Settings` object using Pydantic.
- **Default value:** `60` seconds (a conservative limit that balances most tool needs).
- **Validation:** Enforced `ge=0` so negative values are rejected at startup. A value of `0` cleanly disables the timeout mechanism (for backwards compatibility).

2. Core Execution Engine (`src/pocketpaw/tools/registry.py`)

- Implemented `_get_tool_timeout()` , which safely and defensively fetches the configured timeout, guarding against bad runtime configurations (e.g. `None` values) and falling back to a `_DEFAULT_TOOL_TIMEOUT` .
- Wrapped the primary execution call in `asyncio.wait_for(tool.execute(**params), timeout=timeout)` .

3. Graceful Error & Concurrency Handling

- **Timeout Reporting:** Specifically caught `asyncio.TimeoutError` to return a clear, actionable string to the LLM agent ("Error: Tool '...' timed out after Xs"), so the agent knows to try an alternative approach instead of silently failing.
- **Audit Logging:** Emitted specific timeout events to the security audit logger (`action="tool_timeout"` , `status="timeout"`) to track flaky tools in production.
- **Agent Lifecycle Integration:** Ensured `asyncio.CancelledError` is cleanly re-raised. This guarantees that if the agent session itself is shut down, the cancellation propagates correctly to the tools without being swallowed by generic exception handlers.

4. Robust Test Suite (`tests/test_tool_registry.py`)

- Wrote **35 unit tests** across 9 test classes covering all aspects of the timeout logic.
- Simulated tool delays, verified 0-bypass (disabled timeouts), asserted that concurrent tools don't interfere with each other's timeouts, and ensured negative configs default to 0 safely.
- Kept the test suite CI-friendly by utilizing sub-second `asyncio.sleep()` boundaries, allowing all 35 tests to run in just ~1.09 seconds.

Usage Guide & Best Practices

The new per-tool timeout feature is fully configurable out-of-the-box:

Environment Variable Configuration: You can override the default 60-second limit by setting an environment variable before starting the agent: `bash export`

`POCKETPAW_TOOL_TIMEOUT=30`

JSON Configuration (`~/.pocketpaw/config.json`): `json { "tool_timeout": 15 }`

Disabling the Timeout: If you have a specific long-running agent session that requires infinite execution time (e.g. a massive nightly data processing tool), set the timeout to 0:

```
0: json { "tool_timeout": 0 }
```

Conclusion

The per-tool timeout fix introduces a significant reliability upgrade to the PocketPaw framework. It transforms unpredictable tool hangs into observable, trackable, and recoverable events, bridging the gap between a prototype agent and a production-grade orchestration engine.